

Broadcaster Pushers Audit

OpenZeppelin Security - March 16, 2026

Summary

Type: Library

Timeline: From 2026-02-02 → To 2026-02-06

Languages: Solidity

Findings

Total issues: 10 (8 resolved)

Critical: 0 (0 resolved) · High: 0 (0 resolved) · Medium: 0 (0 resolved) · Low: 4 (3 resolved)

Notes & Additional Information

6 notes raised (5 resolved)

Scope

OpenZeppelin performed an audit of the [openintentsframework/broadcaster](#) repository at commit [e329d3a](#). In addition, [pull request #85](#) at commit [0dc6e34](#) was also reviewed.

In scope were the following files:

This site uses cookies to offer you a better user experience. Information about the cookies we use can be found in our Privacy Policy under "Cookie Policy". You can reject all non-essential cookies by choosing "Decline" or set your own preference under "Cookies settings".

[Cookies settings](#)

Accept

Decline

```
└─ linea
  │   └─ LineaBuffer.sol
  │   └─ LineaPusher.sol
└─ optimism
  │   └─ OptimismBuffer.sol
  │   └─ OptimismPusher.sol
└─ scroll
  │   └─ ScrollBuffer.sol
  │   └─ ScrollPusher.sol
└─ zksync
    └─ ZkSyncBuffer.sol
    └─ ZkSyncPusher.sol
```

Update: The fixes for the findings highlighted in this report have all been merged at commit [4cd580d](#).

System Overview

The Block Hash Pusher system enables trustless propagation of Ethereum L1 block hashes to L2 chains. This infrastructure supports cross-chain verification workflows by making historical L1 block data available on child chains, allowing applications to verify L1 state without relying on external oracles.

The architecture follows a push model where L1 contracts collect recent block hashes and transmit them to L2 buffer contracts via chain-specific cross-chain messaging bridges. Once stored on the L2, these block hashes can be queried by any contract or application that needs to verify L1 state.

Pusher Contracts

Pusher contracts are deployed on Ethereum L1 and serve as the entry point for block hash propagation. The `BlockHashArrayBuilder` base contract provides shared functionality for constructing arrays of recent block hashes using OpenZeppelin's `Blockhash` utility, which handles the EIP-2935 history storage window (8191 blocks).

Each chain has a dedicated pusher implementation:

- `LineaPusher` sends messages via Linea's `IMessageService.sendMessage`.
- `ScrollPusher` sends messages via Scroll's `IL1ScrollMessenger.sendMessage`.
- `ZkSyncPusher` sends messages via ZKsync's `IMailbox.requestL2Transaction`.
- `OptimismPusher` sends messages via Optimism's `ICrossDomainMessenger.sendMessage`.

Pushers are permissionless. Any caller can invoke `pushHashes` with the appropriate payment (where required) to cover L2 execution costs, while Optimism expects `msg.value == 0`.

Buffer Contracts

Buffer contracts are deployed on L2 chains and store the received block hashes in a sparse circular buffer structure. The `BaseBuffer` abstract contract implements a fixed-size circular buffer (393,168 slots, approximately 54 days of L1 history at 12-second block times) using modulo-based indexing for efficient storage and eviction.

Each chain-specific buffer implementation adds access control appropriate to its messaging bridge:

- `LineaBuffer` validates that calls originate from the `L2MessageService` with the correct L1 sender.
- `ScrollBuffer` validates that calls originate from the `L2ScrollMessenger` and checks the cross-domain message sender.
- `ZkSyncBuffer` validates that the caller is the aliased address of the L1 pusher (ZKsync applies address aliasing to L1 senders).
- `OptimismBuffer` validates that calls originate from the `L2CrossDomainMessenger` and checks the cross-domain message sender.

Security Model and Trust Assumptions

The Block Hash Pusher system relies on several external components and makes implicit trust assumptions about the underlying infrastructure:

- **Bridge Security:** The system inherits the security properties of each L2's native messaging bridge (`Linea MessageService`, `Scroll L1/L2 Messengers`, `ZKsync Mailbox`, `Optimism L2CrossDomainMessenger`). Compromise or malfunction of these bridges could result in incorrect or missing block hash data on the L2.
- **Pusher-Buffer Binding:** Each buffer is immutably bound to a single pusher address at deployment. There is no mechanism to update the authorized pusher, requiring redeployment if the pusher address changes.
- **Buffer Eviction Model:** The circular buffer design means older entries are overwritten when the buffer cycles. Applications consuming this data must account for the limited retention window.
- **No Privileged Administration:** The contracts contain no owner roles or administrative functions. Once deployed, the pusher-buffer relationship is fixed and the system operates autonomously.
- **Liveness Dependency:** Block hash availability on L2 depends on external actors calling `pushHashes` and paying the associated cross-chain messaging fees. The system does not guarantee data freshness or completeness.
- **Bridge Message Delivery:** For Linea, message claiming on L2 may require manual intervention if the postman service does not process expensive messages. Scroll supports permissionless replays for failed messages. ZKsync delivery depends on its mailbox/priority queue execution. Optimism messages are relayed by the `L2CrossDomainMessenger` and failed executions can be replayed permissionlessly with a higher gas limit. Thus, delivery depends on relayers and adequate gas settings.

Low Severity

Missing Zero-Address Check in Linea and Scroll Buffer Constructors

The `LineaBuffer` and `ScrollBuffer` constructors do not validate that the `pusher_` argument is non-zero, while `ZkSyncBuffer` enforces this check at deployment time. Although both Linea and Scroll buffers perform this validation at runtime within their `receiveHashes` functions, the absence of a constructor check allows deployment of a non-functional buffer that would revert on every call. Similarly, zero address checks are missing in the constructors of `LineaPusher`, `ScrollPusher` and `ZkSyncPusher`.

Consider adding a zero-address check for the pusher argument in the constructors of `LineaBuffer` and `ScrollBuffer` to fail fast at deployment and maintain consistency across all buffer implementations. Likewise, consider including similar validations for the constructors of `LineaPusher`, `ScrollPusher`, and `ZkSyncPusher`. Additionally, consider removing this check within the execution of each `receiveHashes` function, as the value of the `_pusher` account would already be checked upon deployment.

Update: Resolved in [pull request #68](#). The OpenZeppelin development team stated:

We have added a zero-address check to the constructors of Linea and Scroll Buffers. Similar checks have also been added to `ZkSyncPusher`, `ZkSyncBuffer`, and `LineaPusher` to avoid performing these checks at execution time.

Linea Pusher Allows Value Mismatch Creating Permanently Unclaimable Messages

The `LineaPusher.pushHashes` function forwards the full `msg.value` to the Linea messenger while only decoding `_fee` from the transaction data. In Linea's L1 message service, the Linea bridge forwards `msg.value - _fee` to the destination contract. On L2, a postman or user calls `claimMessage` on the `L2MessageService` with the same `_value` and `_calldata`. Since `LineaBuffer.receiveHashes` is -payable (per the `IBuffer` interface), any call where `msg.value > _fee` creates a message that permanently reverts on claim. This burns the paid fees, leaves the block hashes unapplied, and contradicts documentation stating that `msg.value` must be "at least" the fee.

Consider validating `msg.value == _fee` in `LineaPusher.pushHashes` and reverting with `IncorrectMsgValue` on a mismatch.

Update: Resolved in [pull request #79](#). The OpenZeppelin development team stated:

We have updated the `LineaPusher` contract to validate that `msg.value == _fee` and revert in case there is a mismatch.

Buffer Accepts Non-Monotonic Batches and Emits Misleading Events

The `IBuffer.receiveHashes` NatSpec comment specifies that "the last block in the buffer must be less than the last block being pushed", implying strictly monotonic pushes. However, `BaseBuffer._receiveHashes` silently skips writes for any `blockNumber <= existingBlockNumber` and unconditionally emits `BlockHashesPushed(firstBlockNumber, lastBlockNumber)` regardless of whether any hashes were stored.

Since pushers are permissionless, third parties can enqueue duplicate or out-of-order batches. If the buffer contains hashes up to block 100 and a message arrives for blocks 90-99, all writes are skipped yet `BlockHashesPushed(90, 99)` is emitted. Off-chain systems monitoring these events may misinterpret them as progress. In addition, block 0 is permanently unwritable because `_blockNumberBuffer` defaults to zero and the skip condition always triggers.

Consider emitting `BlockHashesPushed` only when at least one hash is stored, and consider rejecting `firstBlockNumber == 0`.

Update: Resolved in [pull request #80](#). The OpenZeppelin development team stated:

We have updated the `BaseBuffer` contract to verify if at least one block hash is written to storage in order to emit the event and also revert if `firstBlockNumber == 0`.

Batch Size Limit May Cause Unprocessable L2 Messages and Does Not Guarantee Block Hash Availability

The `MAX_BATCH_SIZE` constant is set to 8191 to match the EIP-2935 history window, implying that it prevents requests for stale block hashes. However, this check is ineffective since callers specify `firstBlockNumber` independently. A caller could request hashes starting from an old block number that falls outside the available window, causing `_blockHash` to revert. A check could be performed earlier in `_buildBlockArray` comparing `firstBlockNumber` with the oldest allowed block.

The conflation of batch size with the history window also creates L2 execution risks. A maximum-size batch produces excessively large calldata and requires up to three storage writes per hash in `_receiveHashes`. This may be unprocessable on certain L2s, as zkSync explicitly [rejects transactions exceeding 128kb](#). Since pushers are permissionless, an attacker could submit oversized batches that fail on L2 and potentially block subsequent messages if the bridge processes them in order.

Consider introducing per-chain batch size limits based on benchmarked L2 execution costs and enforcing them in both pushers and buffers as well as implementing earlier checks on `firstBlockNumber` without relying on `MAX_BATCH_SIZE` to check that it is recent enough.

Update: Acknowledged, not resolved. The OpenZeppelin development team stated:

Although the `MAX_BATCH_SIZE` constant is used to check for valid batch sizes, it also serves as a way to check for the validity of `firstBlockNumber` (i.e., `firstBlockNumber <= block.number - MAX_BATCH_SIZE`). Since this constant is dependent on the L1 and not the L2s, it was chosen to keep it as is and users of the pushers should be aware of L2

gas limits when pushing hashes. Besides that, the gas limit on L1 is already a bottleneck for sending transactions, since sending a batch close to the limit would be very expensive and possible would revert with an out-of-gas error.

Notes & Additional Information

Redundant Getters

Throughout the codebase, multiple instances of redundant getter functions were identified, where an `external` function had been exposed to read the value of a `private` state variable:

- `_l1ScrollMessenger` within `ScrollPusher`
- `_lineaRollup` within `LineaPusher`
- `_zkSyncDiamond` within `ZkSyncPusher`
- `_pusher` within `ScrollBuffer`
- `_pusher` within `LineaBuffer`
- `_pusher` within `ZkSyncBuffer`
- `_newestBlockNumber` within `BaseBuffer`

Consider removing `public` getter functions and instead setting the state variables to `public`, clarifying intent and simplifying the codebase.

Update: Acknowledged, not resolved. The OpenZeppelin development team stated:

The OpenZeppelin Solidity guidelines state that all storage must be `private` to avoid potential contracts that inherit from the original contracts to have direct access to the storage.

Custom Errors in `require` Statements

Since Solidity [version 0.8.26](#), custom error support has been added to `require` statements. Initially, this feature was only available through the IR pipeline. However, Solidity [version 0.8.27](#) extended support for this feature to the legacy pipeline as well.

Throughout the codebase, multiple instances of `if-revert` statements were identified that could be replaced with `require` statements:

- The

```
if (blockHash == 0) {  
    revert UnknownParentChainBlockHash(parentChainBlockNumber);  
}
```

 statement in `BaseBuffer.sol`

- The `if (blockHashesLength == 0) {
 revert EmptyBlockHashes();
}` statement in `BaseBuffer.sol`
- The `if (batchSize == 0 || batchSize > MAX_BATCH_SIZE()) {
 revert IPusher.InvalidBatch(firstBlockNumber, batchSize);
}` statement in `BlockHashArrayBuilder.sol`
- The `if (firstBlockNumber + batchSize > block.number) {
 revert IPusher.InvalidBatch(firstBlockNumber, batchSize);
}` statement in `BlockHashArrayBuilder.sol`
- The `if (blockHash == 0) {
 revert InvalidBlockNumber(blockNumber);
}` statement in `BlockHashArrayBuilder.sol`
- The `if (l2MessageService_ == address(0)) {
 revert InvalidL2MessageServiceAddress();
}` statement in `LineaBuffer.sol`
- The `if (msg.sender != address(l2MessageServiceCached)) {
 revert InvalidSender();
}` statement in `LineaBuffer.sol`
- The `if (_pusher == address(0)) {
 revert InvalidPusherAddress();
}` statement in `LineaBuffer.sol`
- The `if (l2MessageServiceCached.sender() != _pusher) {
 revert SenderMismatch();
}` statement in `LineaBuffer.sol`
- The `if (buffer == address(0)) {
 revert InvalidBuffer(buffer);
}` statement in `LineaPusher.sol`
- The `if (l2ScrollMessenger_ == address(0)) {
 revert InvalidL2ScrollMessengerAddress();
}` statement in `ScrollBuffer.sol`
- The `if (msg.sender != address(l2ScrollMessengerCached)) {
 revert InvalidSender();
}` statement in `ScrollBuffer.sol`
- The `if (_pusher == address(0)) {
 revert InvalidPusherAddress();
}` statement in `ScrollBuffer.sol`
- The `if (l2ScrollMessengerCached.xDomainMessageSender() != _pusher) {
 revert DomainMessageSenderMismatch();
}` statement in `ScrollBuffer.sol`
- The `if (buffer == address(0)) {
 revert InvalidBuffer(buffer);
}`

- The `} statement in ScrollPusher.sol`
- The `if (pusher_ == address(0)) {
 revert InvalidPusherAddress();
}` statement in `ZkSyncBuffer.sol`
- The `if (msg.sender != aliasedPusher()) {
 revert NotPusher();
}` statement in `ZkSyncBuffer.sol`
- The `if (_pusher == address(0)) {
 revert InvalidPusherAddress();
}` statement in `ZkSyncBuffer.sol`
- The `if (buffer == address(0)) {
 revert InvalidBuffer(buffer);
}` statement in `ZkSyncPusher.sol`
- The `if (canonicalTxHash == bytes32(0)) {
 revert FailedToPushHashes();
}` statement in `ZkSyncPusher.sol`

For conciseness and gas savings, consider replacing `if-revert` statements with `require` statements.

Update: Resolved in [pull request #73](#) and [pull request #86](#).

Variables Initialized With Their Default Values

Throughout the codebase, multiple instances of variables being initialized with their default values were identified:

- In `BaseBuffer.sol`, the `i` variable
- In `BlockHashArrayBuilder.sol`, the `i` variable

To avoid wasting gas, consider not initializing variables with their default values.

Update: Resolved in [pull request #70](#).

Lack of Indexed Event Parameters

Throughout the codebase, multiple instances of events not having any indexed parameters were identified:

- The `BlockHashesPushed` event of `IBuffer.sol`
- The `BlockHashesPushed` event of `IPusher.sol`

To improve the ability of off-chain services to search and filter for specific events, consider [indexing](#)

event parameters.

Update: Resolved in [pull request #71](#).

Prefix Increment Operator (`++i`) Can Save Gas in Loops

Using the prefix increment operator (`++i`) for loop iteration skips storing the value before the incremental operation, as the return value of the expression is ignored.

Throughout the codebase, multiple opportunities where this optimization could be applied were identified:

- The `++i` in `BaseBuffer.sol`
- The `++i` in `BlockHashArrayBuilder.sol`

Consider using the prefix increment operator (`++i`) instead of the postfix increment operator (`i++`) in order to save gas.

Update: Resolved in [pull request #72](#).

Missing Security Contact

Providing a specific security contact (such as an email address or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is quite beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, eliminating the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

Throughout the codebase, multiple instances of contracts not having a security contact were identified:

- The `BaseBuffer` abstract contract
- The `BlockHashArrayBuilder` abstract contract
- The `IBuffer` interface
- The `IPusher` interface
- The `LineaBuffer` contract
- The `LineaPusher` contract
- The `ScrollBuffer` contract
- The `ScrollPusher` contract
- The `ZkSyncBuffer` contract
- The `IMailbox` interface
- The `ZkSyncPusher` contract

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the `@custom:security-contact` convention is recommended as it has been adopted by the [OpenZeppelin Wizard](#) and the [ethereum-lists](#).

Update: Resolved in [pull request #81](#).

Conclusion

The Block Hash Pusher system provides infrastructure for propagating Ethereum L1 block hashes to L2 chains via native cross-chain messaging bridges. The architecture is intentionally minimal, with permissionless pushers, immutable pusher-buffer bindings, and no administrative functions.

No critical- or high-severity issues were identified. The primary finding involves the Linea pusher's handling of `msg.value`, where excess value over the declared fee creates permanently unclaimable messages. Additional findings address constructor validation inconsistencies, misleading event emissions for skipped batches, and potential L2 execution risks from unbounded batch sizes. Overall, the codebase is well-structured with clear separation between the shared buffer logic and chain-specific access control. The trust assumptions and liveness dependencies are appropriately documented.

The OpenZeppelin development team demonstrated strong technical proficiency and remained responsive throughout the engagement, providing the audit team with clear documentation of the system's intended behavior and trust model provided.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core

system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system. These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.

Ready to secure your code?

[Request an Audit → →](#)

Smart Contracts

Contracts Libraries

Contracts Wizard ↗

Contracts MCP ↗

Upgrades Plugin ↗

Open Source Tools

Relayer

Monitor

UI Builder ↗

Safe Utils ↗

Role Manager ↗

The Open Source Stack

Blockchains

Arbitrum

Base

Ethereum

Linea

Midnight

OP Mainnet

Polkadot

Starknet

Stellar

Zama

ZKsync

Services

Smart Contract Security Audit

Blockchain Infrastructure Audit

ZKP Audit

Security Services

Company

About Us

Why OpenZeppelin

News

Customer Stories

Careers

Brand Kit

Trust ↗

Terms & Policies

Terms of Service

Privacy Policy

Trademark Guidelines

AGPL License

Resources

Research

Documentation ↗

Stats

Forum ↗

[Ethernaut CTF ↗](#)

[Solutions](#)

[Ecosystem Stack](#)

[Financial Institutions Stack](#)



© 2026 Zeppelin Group Ltd